UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ORGANIZACION DE LENGUAJES Y COMPILADORES 1
CATEDRÁTICO: ING. MARIO BAUSTISTA
TUTOR ACADÉMICO: CRISTIAN AXPUAC



MANUAL DE USUARIO DE: LENGUAJE OBJ-C

INTEGRANTES: JENNIFER ANAHI VIELMAN MONZON
ARELIS ESTHER OROZCO HERNÁNDEZ
DOUGLAS ALBERTO AJÚ MUCÍA
OTTONIEL FABRICIO VÁSQUEZ PINEDA

SECCIÓN: P

GUATEMALA, 27 de Junio del 2025

ÍNDICE

OBJETIVOS DEL SISTEMA 3
GENERAL 3
ESPECÍFICOS 3
INTRODUCCIÓN 4
INFORMACIÓN DEL SISTEMA 5
Seleccionar Archivo 5
Cargar archivo 5
Interpretar 5
Árbol de Sintaxis Abstracta (AST) 5
REQUISITOS DEL SISTEMA 6
Hardware 6
Software 6
Dependencias 6
FLUJO DE LAS FUNCIONALIDADES DEL SISTEMA 6
Inicio de la aplicación 7
Seleccionar Archivo 7
Cargar Archivo 9
Visualización del Archivo 9
Interpretar 10
Botón de Reporte de Errores 10
Botón de Tabla de Símbolos 11
Botón de AST 12
Botón Reporte de Vectores 13

Botón Volver 14
CONCLUSIÓN.....15

OBJETIVOS DEL SISTEMA

GENERAL

Proporcionar al usuario una guía clara y detallada para explorar todas las funcionalidades del proyecto OBJ-C, desde la carga del .objc, la interpretación y los diferentes reportes con los que se cuenta en el proyecto.

ESPECÍFICOS

- ✓ Describir paso a paso el proceso de carga e interpretación de los archivos
- ✓ Explicar cómo ejecutar el análisis léxico, sintáctico y semántico, interpretar resultados y corregir errores.
- ✓ Detallar la forma de generar y consultar los reportes de errores, símbolos y AST.
- ✓ Implementación de lenguajes imperativos como declaración de variables, expresiones aritméticas y lógicas, estructuras de control,
- ✓ Manejo avanzado de vectores n-dimensionales, procedimientos y conjunto de funciones nativas.
- ✓ Ofrecer una interfaz intuitiva con botones claramente etiquetados.

INTRODUCCIÓN

El proyecto de lenguaje OBJ-C es una construcción de compiladores e intérpretes, a través de una sintaxis que combina declaraciones, variables escalares, manejo de estructuras de control clásico (for, while, do while) y soporte para vectores multidimensionales, OBJ-C permite al usuario describir programas. En la segunda dase se incorporó procedimientos con ámbitos independientes y un conjunto de funciones nativas para ilustrar como se integran directamente en el lenguaje.

El compilador esta diseñado en Python y se utiliza la biblioteca PLY (Python Lex Yacc) para llevar a cabo el análisis léxico y sintáctico, así como un riguroso chequeo semántico que valida tipos, rangos de vectores y formas de procedimientos. La interfaz web, construida con Flask ofrece un editor de texto interactivo donde se carga o edita el código .objc, botones para interpretar el Código y generar reportes (errores, símbolos, vectores) y un módulo que grafica automáticamente el árbol de sintaxis abstracta (AST).

INFORMACIÓN DEL SISTEMA

El proyecto de interprete de OBJ-C es una aplicación web desarrollada en Python con Flask que interpreta un Código .objc dando reportes de errores, símbolos y vectores creando de forma gráfica el AST, está compuesto por diferentes funciones que hacen más entendible el programa.

A continuación, se describen sus componentes principales:

Seleccionar Archivo

Permite al usuario poder acceder a su explorador de archivos, donde el usuario puede seleccionar el archivo .objc para interpretar

Cargar archivo

A usar el botón cargar archivo permite que el archivo objc que el usuario tiene seleccionada pueda ser cargado al texto de entrada donde se puede modificar o eliminar parte de ese Código.

Interpretar

Al pulsar el botón "interpretar" lo que hace es que el código que esta cargado en el texto de entrada es leído y muestra su resultado en el texto de salida, ya sea si es interpretado correctamente o tiene errores.

Árbol de Sintaxis Abstracta (AST)

Al momento de que el código es interpretado, también se genera de forma gráfica un AST donde es se puede visualizar de forma más agradable la forma de compilación del código .objc

Reportes

Además del AST el sistema genera otros reportes

- ✓ Reporte de Errores: muestra en una tabla los distintos errores que se pueden generar léxicos, sintácticos y semánticos indicando la línea y columna donde se encuentra el error.

- ✓ Tabla de Símbolos: genera un listado de los símbolos que el código .objc tiene identificado el ID, Entidad, Tipo, Valor, Entorno, línea y columna.

- ✓ Reporte de Vectores: genera un reporte de cada ejercicio que maneja vectores indicando el tipo y las dimensiones, así como la representación original y la representación lineal (Row-major)

Al finalizar la interpretación del código, automáticamente se abre el navegador de Microsoft Edge con el reporte de AST.

REQUISITOS DEL SISTEMA

Hardware

Computadora con al menos 4 GB de RAM y 500 MB de espacio libre en disco.

Software

Sistema operativo Windows, macOS o Linux con Visual Studio Code instalado y Python 3.8 o superior.

Dependencias

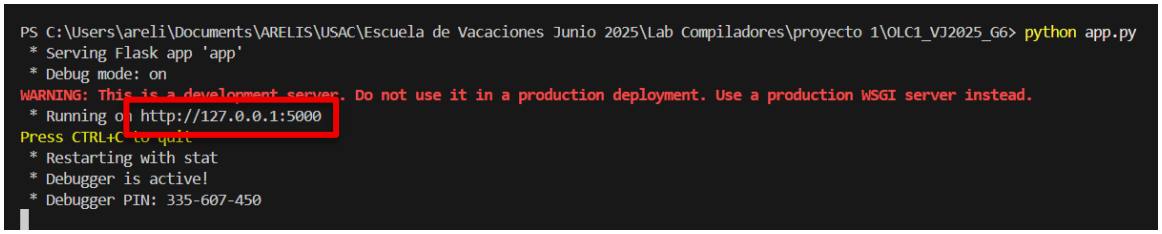
Librerías de ply (Python Lex-Yacc), Flask, Graphviz (modulo python que envuelve la herramienta graphviz) y un navegador web moderno.

FLUJO DE LAS FUNCIONALIDADES DEL SISTEMA

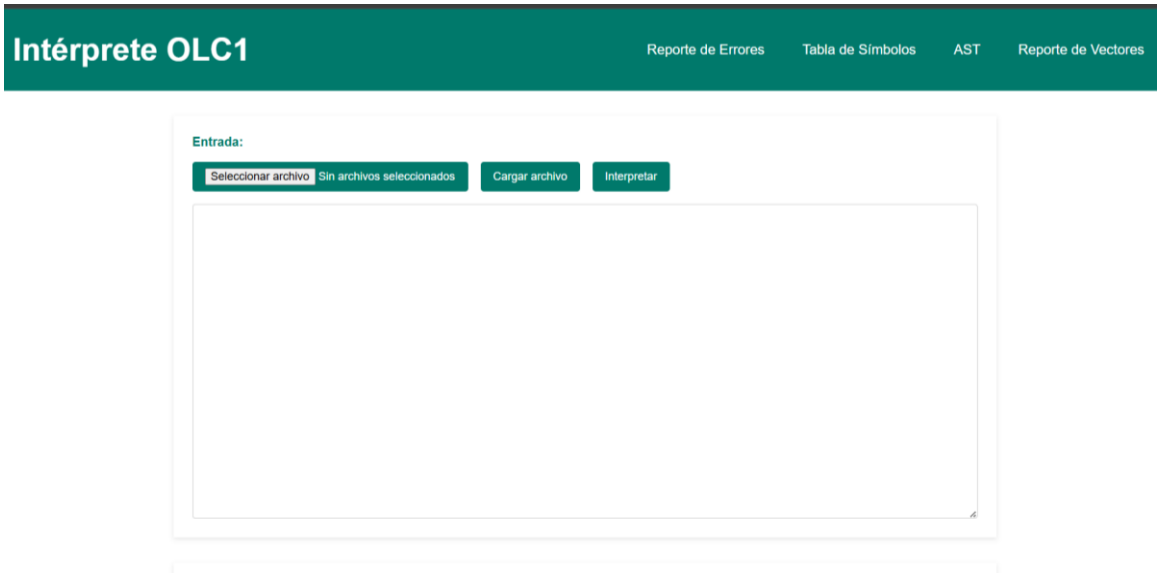
Inicio de la aplicación

Al ejecutar el programa encontramos el puerto del programa que abre el navegador web con la pagina principal del programa donde estarán los botones de seleccionar, cargar e interpretar archivo.

- ✓ Ejecución del programa:



- ✓ Página Principal:

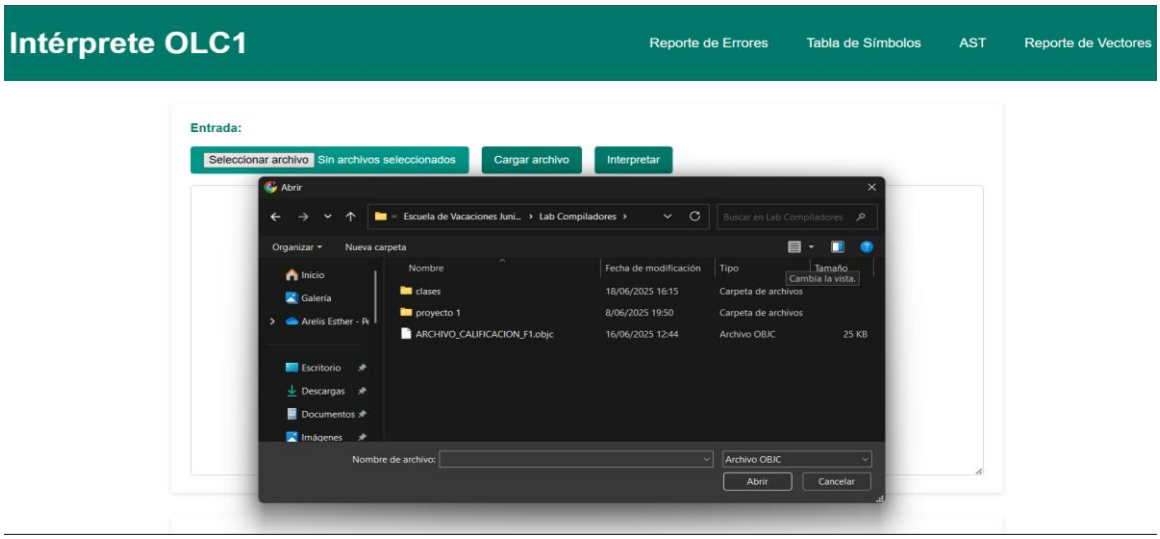


Seleccionar Archivo

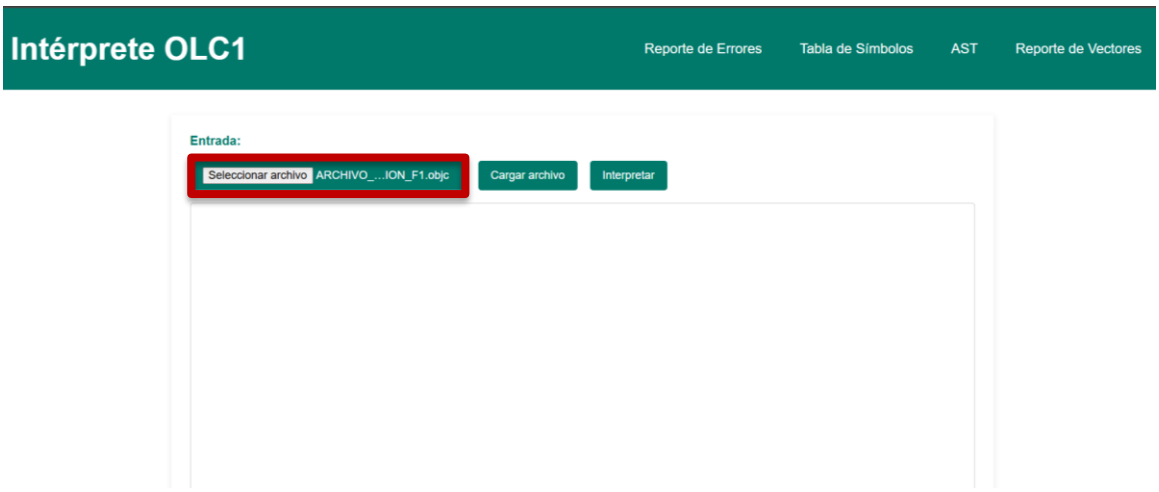
Este botón permite que el usuario seleccione el archivo que quiere interpretar

- ✓ Botón Seleccionar Archivo:





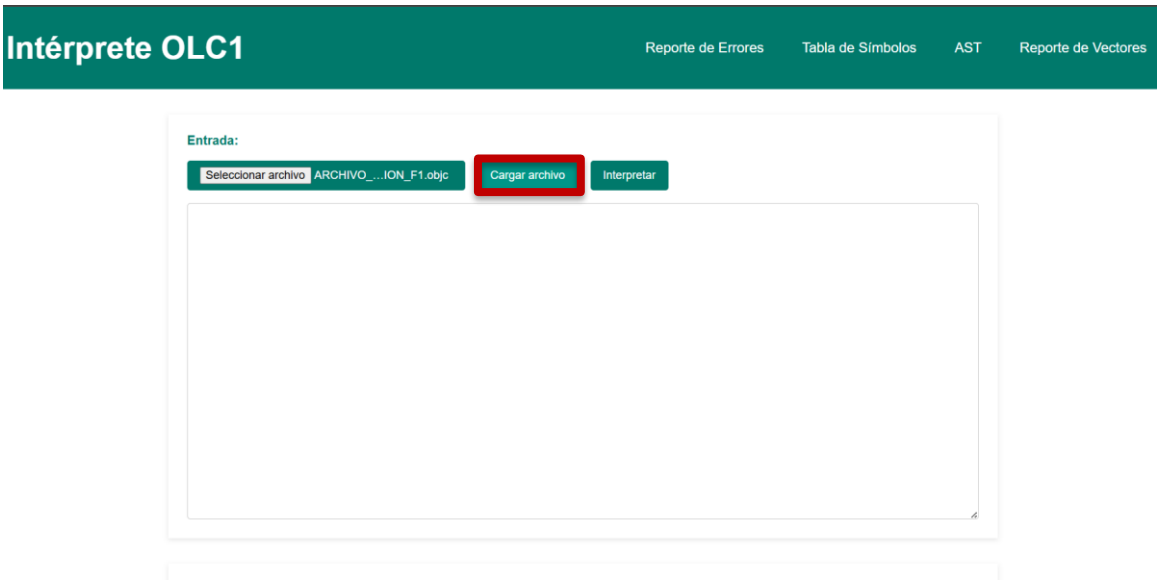
✓ Selección del archivo .objc:



Cargar Archivo

Al seleccionar este botón permite que lo que esta seleccionado sea cargado a la bandeja de entrada

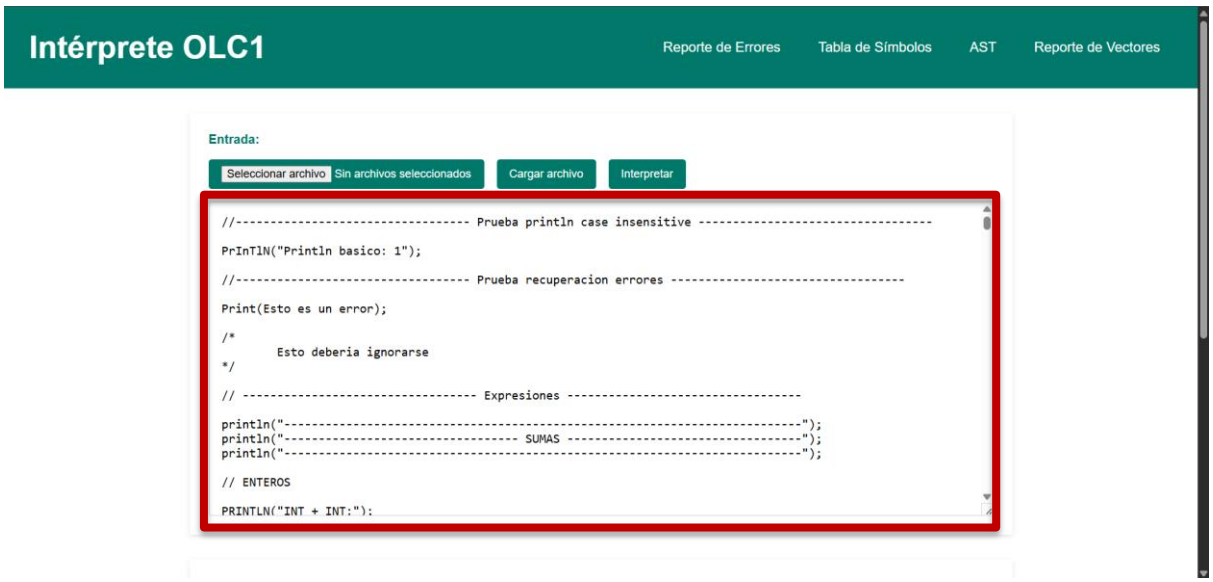
- ✓ Carga de Archivo



Visualización del Archivo

Al cargar el código, este puede ser visualizado en el cuadro de texto de entrada

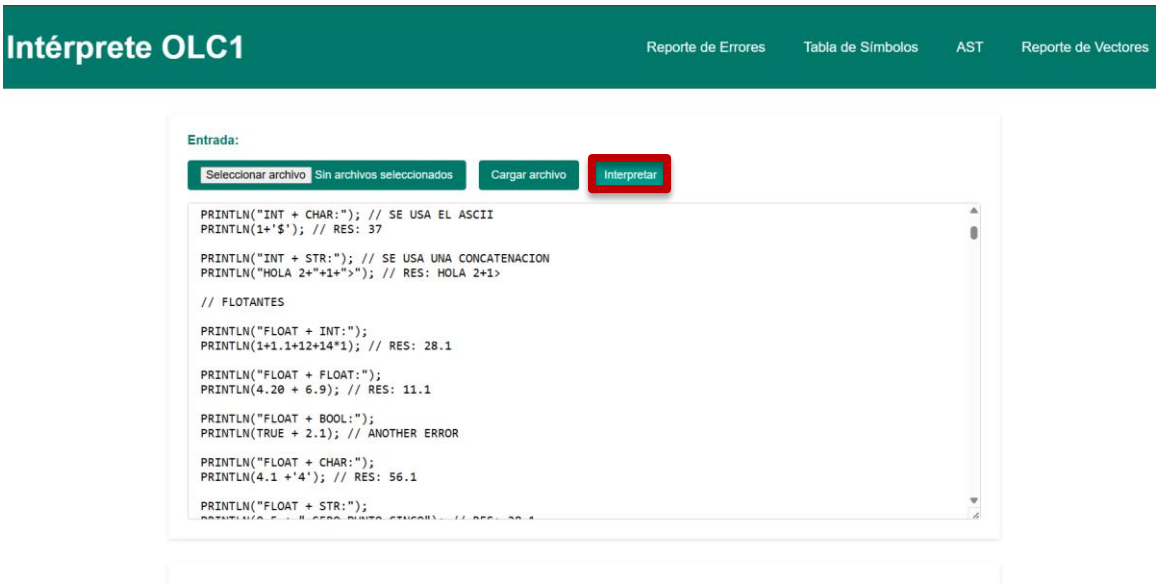
- ✓ Visualización



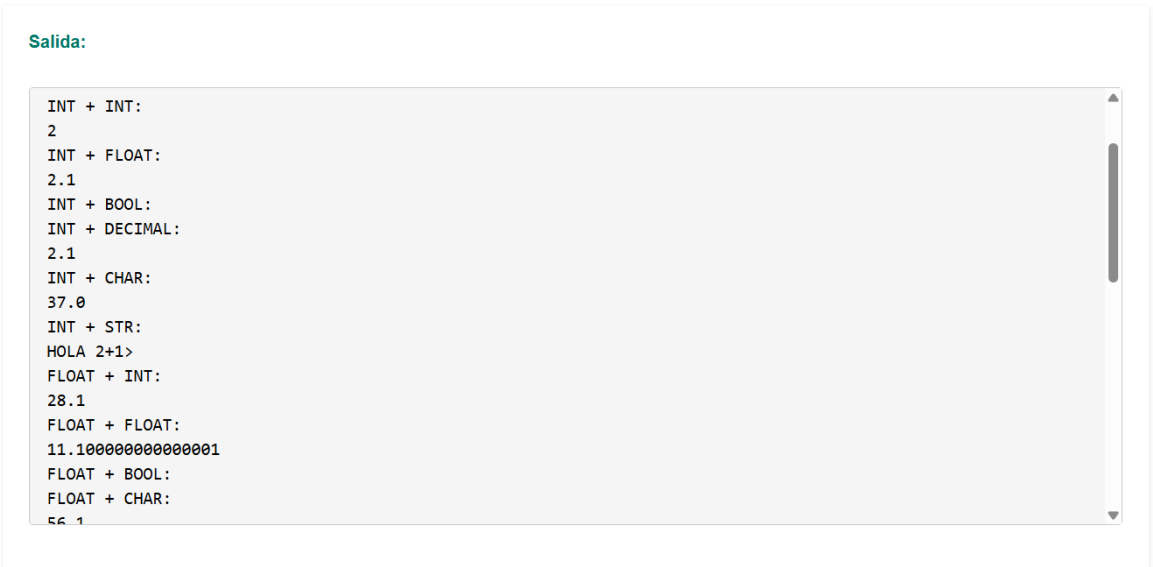
Interpretar

Este botón permite interpretar el código que se encuentra en el texto de entrada.

- ✓ Botón Interpretar



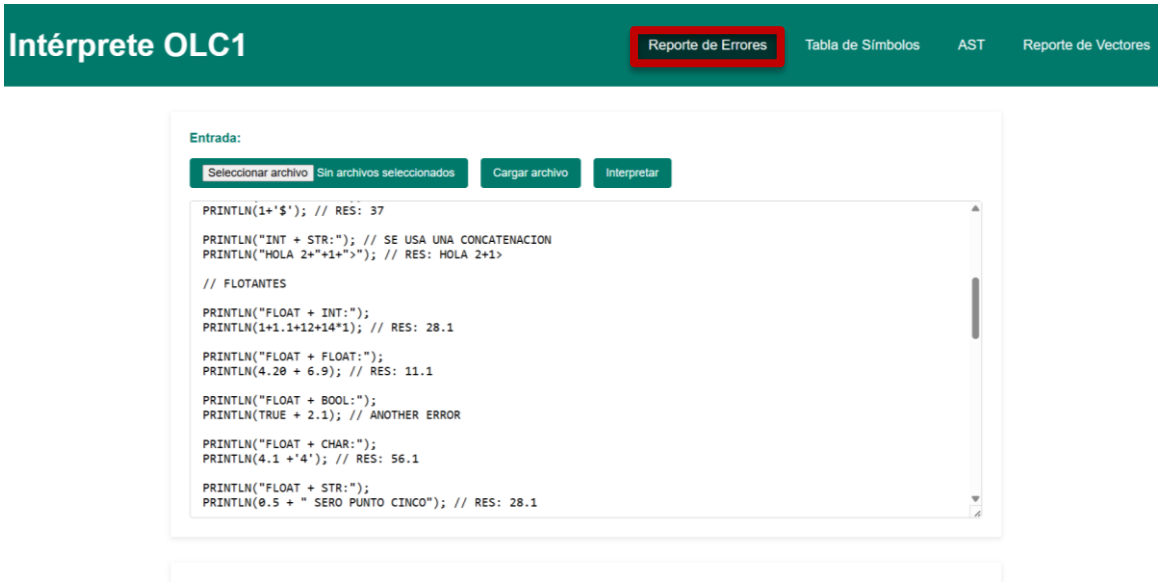
- ✓ Bandeja de Salida

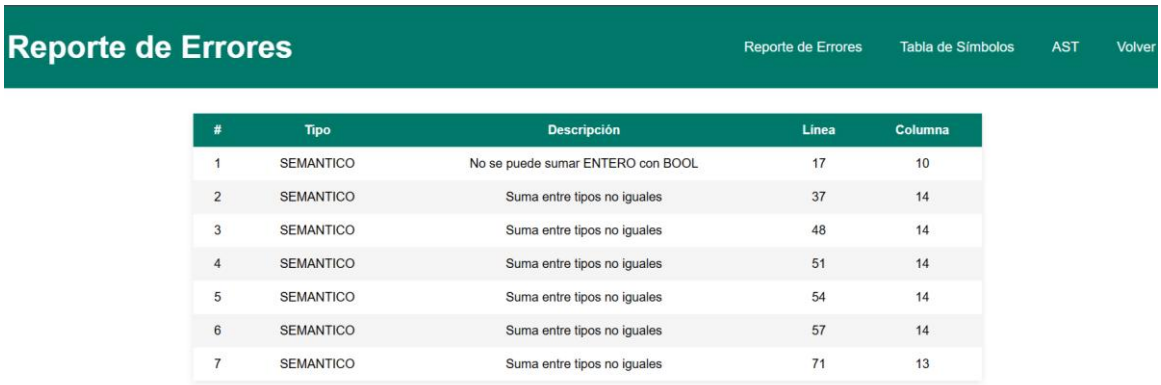


Botón de Reporte de Errores

Este botón permite visualizar los errores que tiene el código ya sea léxico, sintáctico y semántico.

✓ Botón Reporte de Errores:

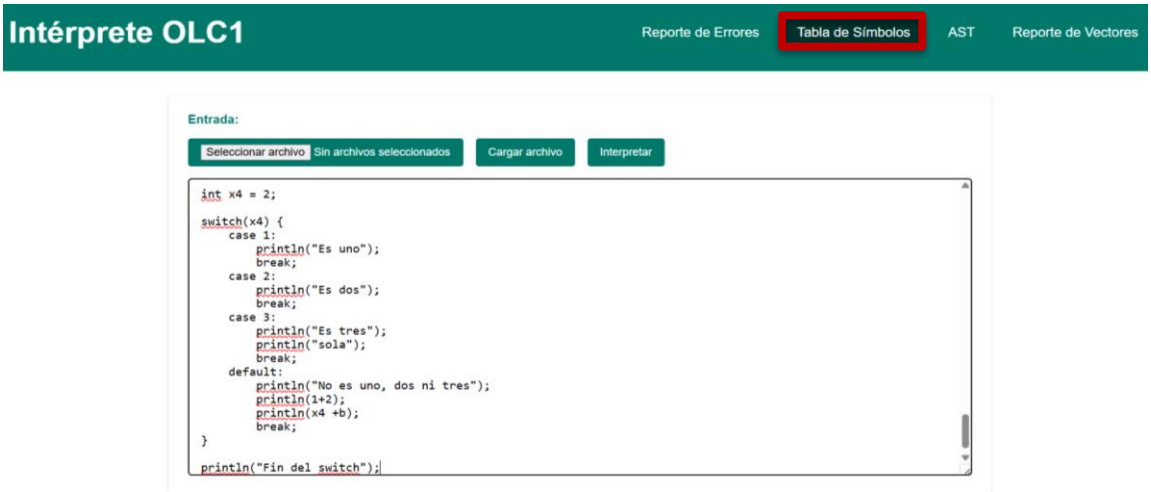


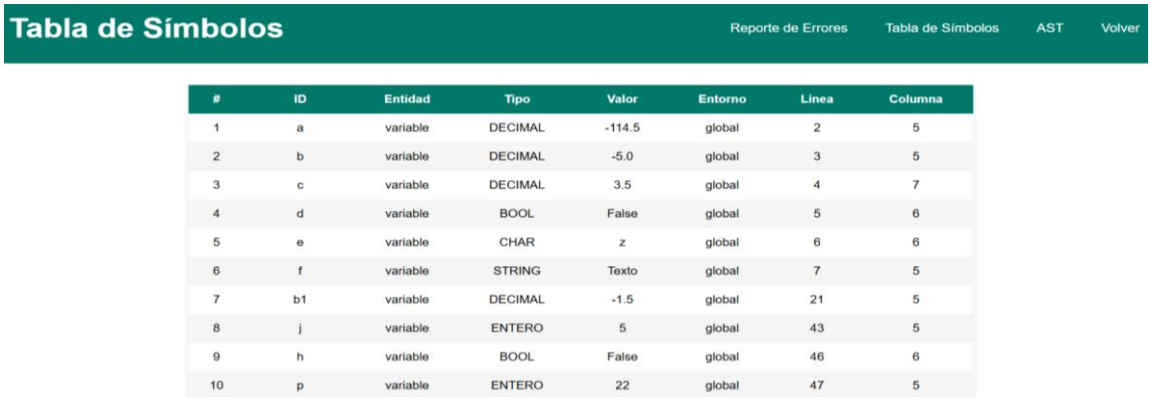


Botón de Tabla de Símbolos

Este botón permite visualizar los símbolos utilizados en el código que se interpreto

✓ Botón Tabla de Símbolos:



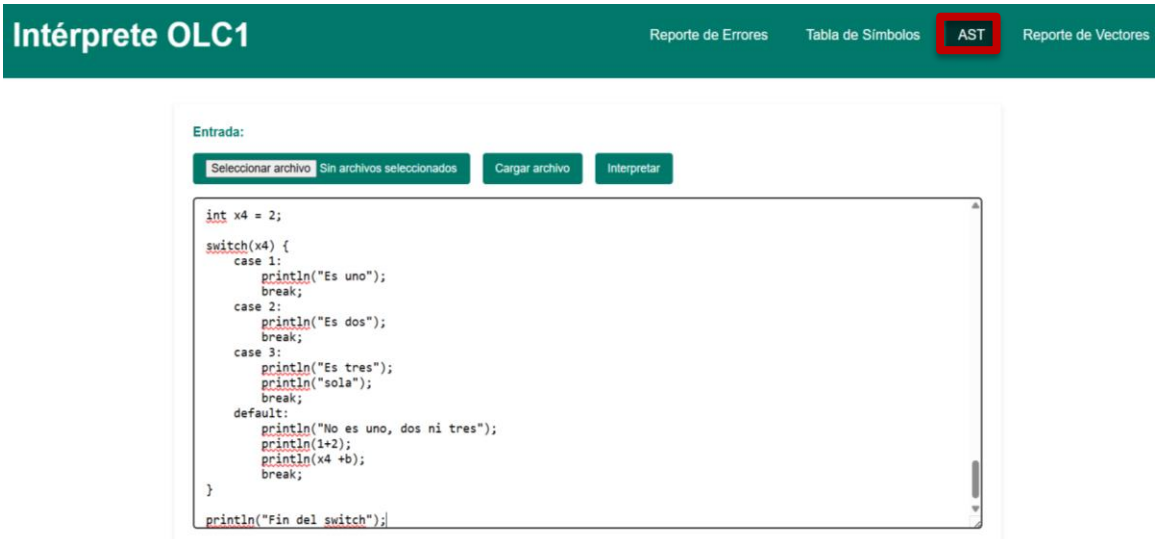


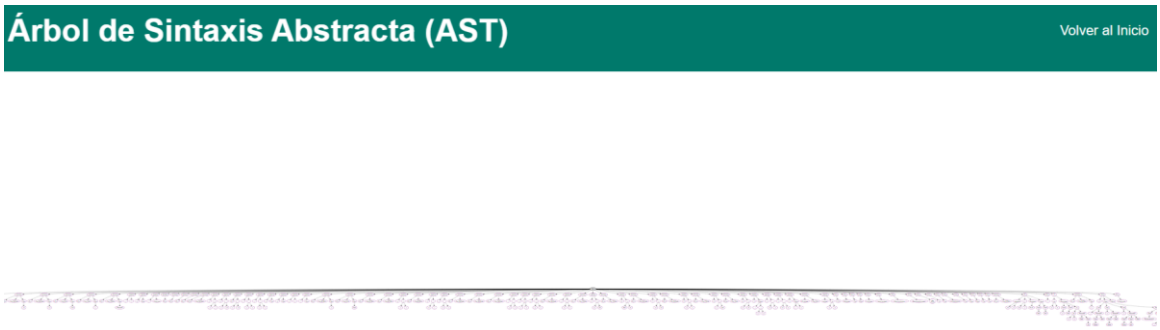
#	ID	Entidad	Tipo	Valor	Entorno	Linea	Columna
1	a	variable	DECIMAL	-114.5	global	2	5
2	b	variable	DECIMAL	-5.0	global	3	5
3	c	variable	DECIMAL	3.5	global	4	7
4	d	variable	BOOL	False	global	5	6
5	e	variable	CHAR	z	global	6	6
6	f	variable	STRING	Texto	global	7	5
7	b1	variable	DECIMAL	-1.5	global	21	5
8	j	variable	ENTERO	5	global	43	5
9	h	variable	BOOL	False	global	46	6
10	p	variable	ENTERO	22	global	47	5

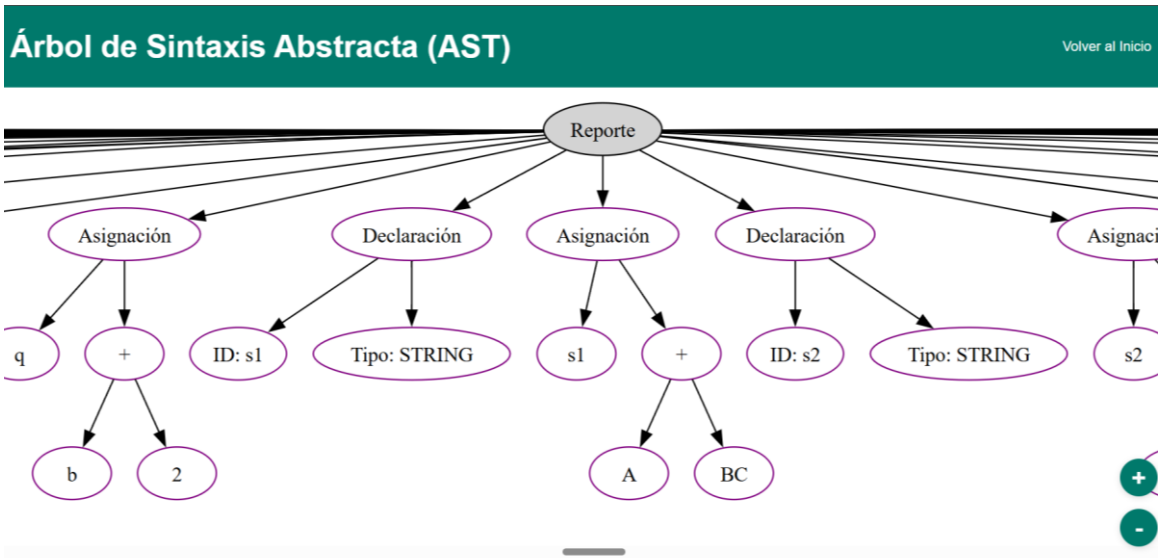
Botón de AST

Este botón permite visualizar el AST generado del codigo .objc

- ✓ Botón AST



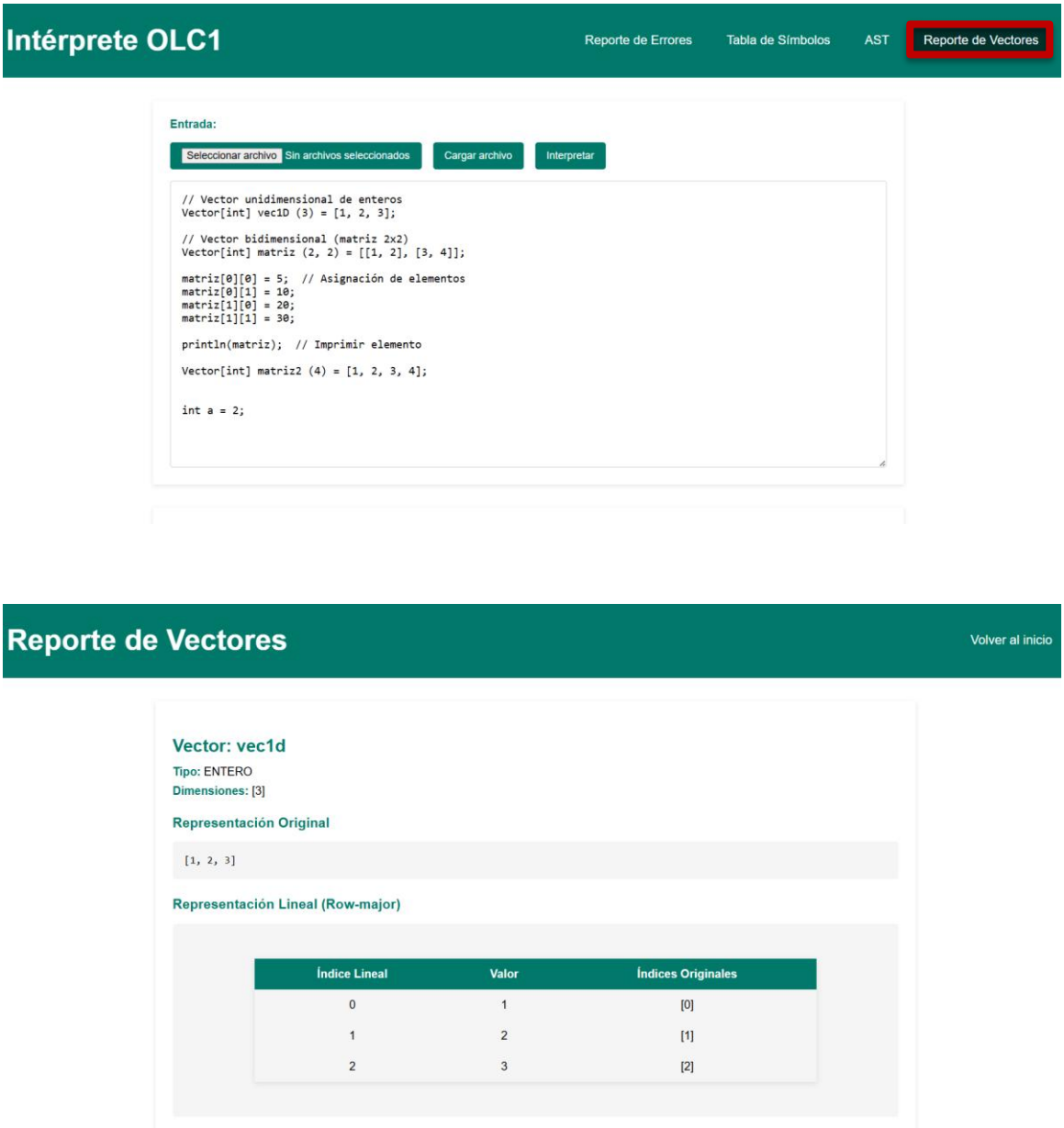




Botón Reporte de Vectores

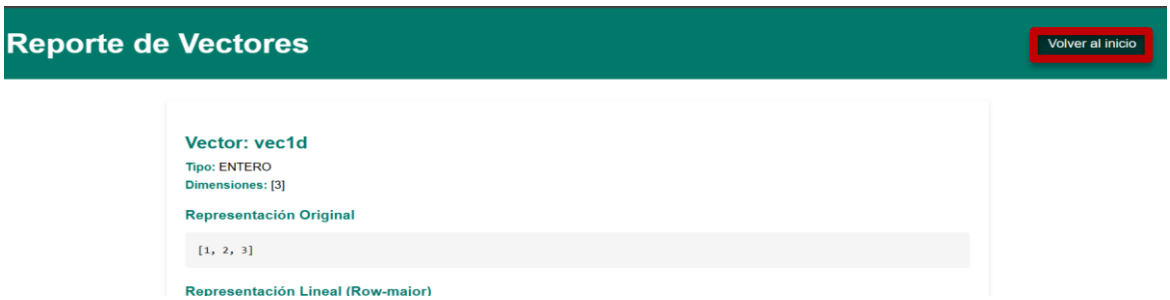
Este reporte genera un reporte de los vectores que se encuentren en el código cargado

- ✓ Botón Reporte de Vectores:



Botón Volver

El botón volver que se puede encontrar en reporte de errores, tabla de símbolos, AST y reporte de vectores, permite que se pueda regresar a la pagina principal del proyecto.



CONCLUSIÓN

El desarrollo del proyecto Object-C ha permitido implementar un entorno funcional e intuitivo para la escritura, análisis y ejecución de código fuente basado en un lenguaje diseñado para el aprendizaje de estructuras sintácticas y semánticas. Gracias a la integración de un analizador léxico y sintáctico robusto, el sistema ha demostrado ser capaz de procesar instrucciones con precisión y reportar errores detalladamente, fomentando una mejor comprensión por parte del usuario.

La generación de AST visuales, la tabla de símbolos, el reporte de errores y la ejecución de código en tiempo real fortalecen la experiencia del usuario al brindar retroalimentación inmediata. Además, la interfaz web desarrollada con Flask simplifica el uso de la herramienta sin necesidad de instalar entornos pesados, haciéndola accesible desde cualquier equipo con navegador moderno.

En conclusión, Obj-C cumple con su propósito como herramienta educativa y técnica para compilar, interpretar y visualizar código, sentando bases sólidas para su extensión futura hacia un compilador más avanzado o una implementación más completa del lenguaje.